

Testing Report

Debate Management System (VIVAATHI)

Final milestone · 25 September 2026

Git branch: Testing (based on main @ f344c79)

Measure	Result
Automated tests / API requests executed	262
Backend line coverage (JaCoCo)	86.9%
Frontend line coverage (Vitest, auth & routing modules)	78.9%
Defects found by testing	15 (3 high, 5 medium, 7 low)
Lighthouse performance score	99–100

Contents

1. Summary.....	3
2. Test strategy	3
2.1 Scope.....	4
2.2 How results are judged	4
3. Tools and frameworks.....	4
4. Results by level	5
4.1 Backend unit tests (JUnit 5 + Mockito).....	5
4.2 Backend integration and access-control tests.....	5
4.3 Frontend component tests (Vitest + React Testing Library)	6
4.4 API end-to-end tests (Postman / Newman)	6
4.5 Browser end-to-end tests (Playwright on Microsoft Edge)	6
4.6 Non-functional: Lighthouse audit (production build, desktop).....	7
5. Code coverage	8
5.1 Backend (JaCoCo)	8
5.2 Frontend (Vitest, V8 coverage)	8
6. Test evidence.....	10
7. Defect log.....	13
7.1 Details and suggested fixes.....	13
7.2 Other observations.....	15
7.3 Bug resolution history	15
8. Manual testing.....	16
9. How to run the tests.....	16
10. Limitations and next steps	16

1. Summary

We tested the Debate Management System at four levels: backend unit tests, backend integration tests (including role-based access control), frontend component tests, and end-to-end tests through both the API (Postman) and a real browser (Playwright). We also ran a Lighthouse audit for performance and accessibility. Everything runs from the command line and produces a report file, listed in Section 9.

The tests check what the system should do, not what it currently does. When a test failed because of a real problem in the application, we kept the test as it was and recorded the problem as a defect. That is why some tests are shown as failing: each failure matches a defect in Section 7.

Level	Tool	Tests	Passed	Failed	Coverage
Backend unit	JUnit 5 + Mockito	46	46	0	85.7% of service layer
Backend integration & RBAC	MockMvc + H2	70	55	15	86.9% overall (lines)
Frontend components	Vitest + RTL	52	52	0	78.9% (lines)
API end-to-end	Postman / Newman	69 requests / 197 checks	181	16	44 of 67 endpoints
Browser end-to-end	Playwright (Edge)	25	21	4	Main user journeys

Main findings. Tournament creation, match scoring, results, leaderboards, statistics, connections, messaging and the multi-tab session handling all work end-to-end. Role checks do stop debaters and judges from performing organizer actions. Testing found 15 defects. The most important are: (1) users can edit other users' profiles and post comments under other people's names; (2) judges never see the organizer's custom score criteria, because the template endpoint fails; and (3) expired logins are never detected by the frontend, because the backend returns 403 instead of 401.

2. Test strategy

We followed the testing pyramid: many fast, isolated tests at the bottom and fewer, slower tests that exercise the whole system at the top.

Level	What it proves	Environment
Unit	Business rules in each service (scoring, winners, judge codes, connections, JWT) are correct in isolation	Mockito fakes, no database, ~2 s
Integration	Controllers, security filter, JPA and database work together; every organizer-only endpoint is protected	Full Spring Boot app, in-memory H2 (PostgreSQL mode)
Component	Login, sign-up, role selection, protected routes, session storage and API client behave correctly	jsdom, API mocked
API end-to-end	A realistic scenario across 44 of the 67 endpoints works against the real server and PostgreSQL	Backend jar on :8081, PostgreSQL database dms_test
Browser end-to-end	Real users can complete key journeys in the real UI	Vite frontend on :5174 + backend + dms_test, Microsoft Edge
Non-functional	Performance, accessibility, best practice, SEO	Production build, Lighthouse desktop preset

2.1 Scope

- In scope: authentication, roles (DEBATER, JUDGE, ORGANIZER), tournaments, matches, score sheets, results and statistics, connections, messages, diaries, calendar, news, notifications, search, newsletter.
- Test data: every run creates its own users with unique names, so results do not depend on leftover data. End-to-end runs use a separate PostgreSQL database (dms_test), never the development database.
- Out of scope: load testing, testing of the deployed Azure/Vercel environment, and real user-acceptance sessions. A manual test sheet for the team is provided instead (Section 8).

2.2 How results are judged

- Each test states the correct behavior, such as "a debater must get 403 Forbidden". A failing test becomes a defect entry. The test is not weakened to make it pass.
- Where the application blocks an action but answers with the wrong HTTP status code, the test records two things separately: whether the action is blocked (security) and whether the right status code is used (API correctness). This keeps working security separate from a cosmetic code problem.

3. Tools and frameworks

Tool	Used for	Why we chose it
JUnit 5	Backend unit & integration test framework	Standard for Spring Boot; already included via spring-boot-starter-test
Mockito	Replaces repositories with fakes in unit tests	Tests business rules in isolation, in milliseconds
Spring MockMvc + spring-security-test	Calls real controllers through the real security filter chain	Verifies RBAC and HTTP status codes without starting a server
H2 (PostgreSQL mode)	In-memory database for integration tests	Fast, repeatable, never touches real data
JaCoCo	Backend code-coverage report	Shows which lines and branches the tests exercised
Vitest + React Testing Library	Frontend component & logic tests	Native to Vite; tests the UI the way a user sees it
@vitest/coverage-v8	Frontend coverage report	Built into the Vitest toolchain
Postman + Newman	API end-to-end scenario, run from the command line	The collection can be opened in Postman for a live demo; Newman produces HTML/JUnit reports
Playwright (Microsoft Edge)	Browser end-to-end tests with screenshots and traces	Drives the real UI against the real backend and database
Google Lighthouse	Performance, accessibility, best-practice and SEO audit	Industry-standard non-functional quality scores
GitHub Actions	Runs the suites automatically on every push	Keeps tests running after the presentation

4. Results by level

4.1 Backend unit tests (JUnit 5 + Mockito)

Unit tests cover the business rules. All of them pass.

Test class	What is tested	Tests	Failed
JwtUtilTest	Token creation; rejection of expired, tampered, foreign-secret and garbage tokens	5	0
AuthServiceTest	Sign-up per role (stats rows), duplicate username/e-mail, login by username/e-mail, wrong password, no user enumeration	10	0
ScoreSheetServiceTest	Judge identity checks, double submission, winner by average score, tie rule, best-speaker vote, stats update, reopen	8	0
TournamentAndMatchServiceTest	Tournament creation with schools/judges/template, duplicate debater, judge codes, match round numbers, next-round pairing	10	0
SocialServicesTest	Connection states, blocking, diary likes/verify/delete permissions, leaderboard points	13	0

4.2 Backend integration and access-control tests

These tests start the whole Spring Boot application with an in-memory database and call it over HTTP using real login tokens.

Test class	What is tested	Tests	Failed
AuthApiIntegrationTest	Sign-up/login API, /me, validation, newsletter	8	1
RbacIntegrationTest	Organizer-only matrix, public vs private endpoints, forged and expired tokens, status codes	47	5
TournamentWorkflowIntegrationTest	Full lifecycle: tournament → match → two judges score → winner, stats, leaderboard; social features; 404s	7	2
OwnershipSecurityIntegrationTest	Acting on another user's profile, comments, notifications, tournament; e-mail exposure	8	7

Role-based access control matrix. Each organizer-only endpoint was called by every role. All 28 combinations behave safely: only organizers get through. The footnotes mark where the status code is wrong (DEF-05, DEF-10).

Organizer-only endpoint	Organizer	Judge	Debater	No login
POST /api/tournaments	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²
DELETE /api/tournaments/{id}	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²
POST /api/tournaments/{id}/judges	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²
POST /api/matches	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²
POST /api/tournaments/{id}/generate-next-round	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²
POST /api/score-templates	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²
POST /api/score-sheets/{id}/reopen	Allowed ✓	Blocked ✓ ¹	Blocked ✓ ¹	Blocked ✓ ²

¹ Returns 400 "Access Denied" instead of 403 (DEF-10). ² Returns 403 instead of 401 (DEF-05).

4.3 Frontend component tests (Vitest + React Testing Library)

Test file	What is tested	Tests	Failed
LoginPage.test.tsx	Empty form, server error message, redirect per role, password toggle	7	0
SignupPage.test.tsx	Password mismatch/length, judge-only fields, successful sign-up, username taken, role selection	8	0
App.test.tsx	Protected routes: 7 redirects for visitors, 5 wrong-role refusals, 5 allowed cases, unknown URLs	18	0
AuthContext.test.tsx	Session restore, login/logout, corrupted storage, cross-tab user switch and logout warnings	8	0
axios.test.ts	Bearer token attached; 401 clears session and redirects; 403 does not	4	0
utils.test.tsx	Notification routing, avatar URLs, toast timeout	7	0

All frontend tests pass. The project's only existing frontend test was broken (it looked for an old password placeholder after the UI redesign), so it was rewritten.

4.4 API end-to-end tests (Postman / Newman)

The collection runs 69 requests in order as one realistic scenario. The requests share state, such as tokens and ids, through environment variables. Every request also checks that it responds in under 2 seconds. The same collection can be opened in the Postman app for a live demo.

Folder	Requests	Checks	Failed checks
01 Health & public pages	4	9	1
02 Authentication	13	45	2
03 Role-based access control	5	15	4
04 Tournament lifecycle	21	65	4
05 Social features	21	52	0
06 Ownership & privacy	5	11	5

Result: 181 of 197 checks passed. All 16 failed checks correspond to defects already found by the backend tests, which confirms them against the real PostgreSQL database.

4.5 Browser end-to-end tests (Playwright on Microsoft Edge)

Spec	Test	Result
01-auth	organizer logs in through the UI and lands on their dashboard	Passed
01-auth	judge logs in through the UI and lands on their dashboard	Passed
01-auth	debaterA logs in through the UI and lands on their dashboard	Passed
01-auth	wrong password shows an error and stays on the login page	Passed
01-auth	new user can sign up and is taken to their dashboard	Passed
01-auth	logged-out visitor cannot open the organizer dashboard	Passed
01-auth	debater cannot open the create-tournament page	Passed
01-auth	user can log out from the navbar menu	Passed

Spec	Test	Result
02-session	logging in as another user in a second tab warns the first tab	Passed
02-session	logging out in a second tab tells the first tab the session ended	Passed
02-session	an expired token sends the user back to login	Failed
03-tournament	organizer creates a tournament with the 6-step wizard	Passed
03-tournament	a visitor can find the new tournament with search	Passed
03-tournament	judge opens the score sheet for an assigned match	Passed
03-tournament	judge submits the score sheet	Passed
03-tournament	match result appears on the tournament page	Passed
04-public	/ loads without errors	Passed
04-public	/news loads without errors	Passed
04-public	/about loads without errors	Passed
04-public	/scoring loads without errors	Passed
04-public	search finds seeded debaters	Passed
04-public	home page loads live matches for visitors without API errors	Failed
04-public	layout works on a phone-sized screen	Passed
05-score-sheet	score sheet shows the organizer's custom criteria	Failed
05-score-sheet	after submitting, reopening the sheet shows "already submitted"	Failed

21 of 25 browser tests pass. The 4 failures show defects DEF-03, DEF-04, DEF-05 and DEF-11 from a real user's point of view. Screenshots and traces are saved for every test.

4.6 Non-functional: Lighthouse audit (production build, desktop)

Page	Performance	Accessibility	Best practices	SEO	LCP
Home /	99	94	96	83	0.9 s
Login /login	100	81	100	83	0.7 s
News /news	99	94	100	83	0.9 s
Role select /role-select	99	94	100	83	0.8 s

Performance is excellent on every page. The accessibility and SEO points lost are listed as DEF-15. The home page's best-practice deduction is the console error caused by DEF-11.

5. Code coverage

5.1 Backend (JaCoCo)

Package	Line coverage	Branch coverage
com.dms.service	85.7%	77.1%
com.dms.controller	65.4%	56.3%
com.dms.security	100%	70%
com.dms.dto	99.4%	73.3%
com.dms.entity	100%	n/a
com.dms.exception	72.7%	60%
com.dms.config	100%	n/a
Total	86.9%	74.2%

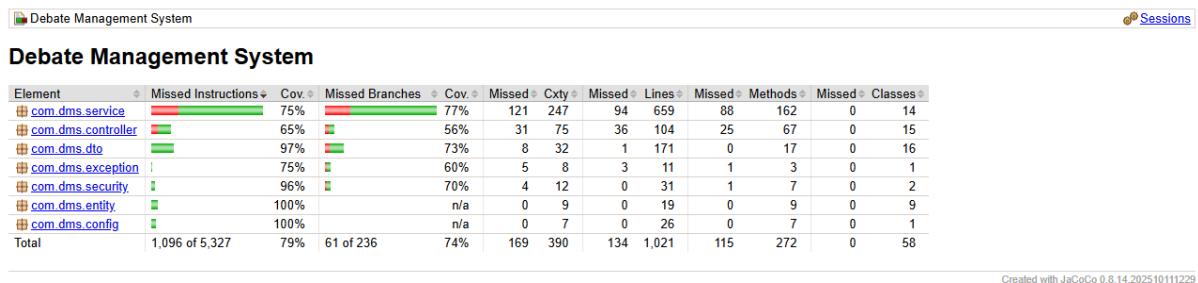


Figure 1. JaCoCo coverage report for the backend (reports/backend/jacoco/index.html)

5.2 Frontend (Vitest, V8 coverage)

Coverage was measured on the modules containing logic: the API client, the auth context, route protection, the auth pages, utilities, the toast and the session modal. Lines 78.9%, statements 76.11%, branches 74.37%, functions 73.97%. Page layouts and dashboards are covered by the Playwright tests instead.

All files

76.11% Statements 264/268 74.37% Branches 119/160 73.97% Functions 54/73 78.9% Lines 187/237

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ^		Statements ^		Branches ^		Functions ^		Lines ^	
src		100%	7/7	100%	7/7	100%	2/2	100%	6/6
src/api		100%	13/13	100%	6/6	100%	3/3	100%	13/13
src/components/common		78.57%	22/28	81.81%	9/11	64.28%	9/14	89.47%	17/19
src/context		96.72%	59/61	75.86%	22/29	100%	14/14	98.27%	57/58
src/pages/auth		76.57%	85/111	73.33%	55/75	72.72%	24/33	78.57%	77/98
src/utlis		37.5%	18/48	62.5%	20/32	28.57%	2/7	39.53%	17/43

Code coverage generated by [istanbul](#) at 2026-09-25T15:37:51.685Z

Figure 2. Frontend coverage report (reports/frontend/coverage/index.html)

6. Test evidence

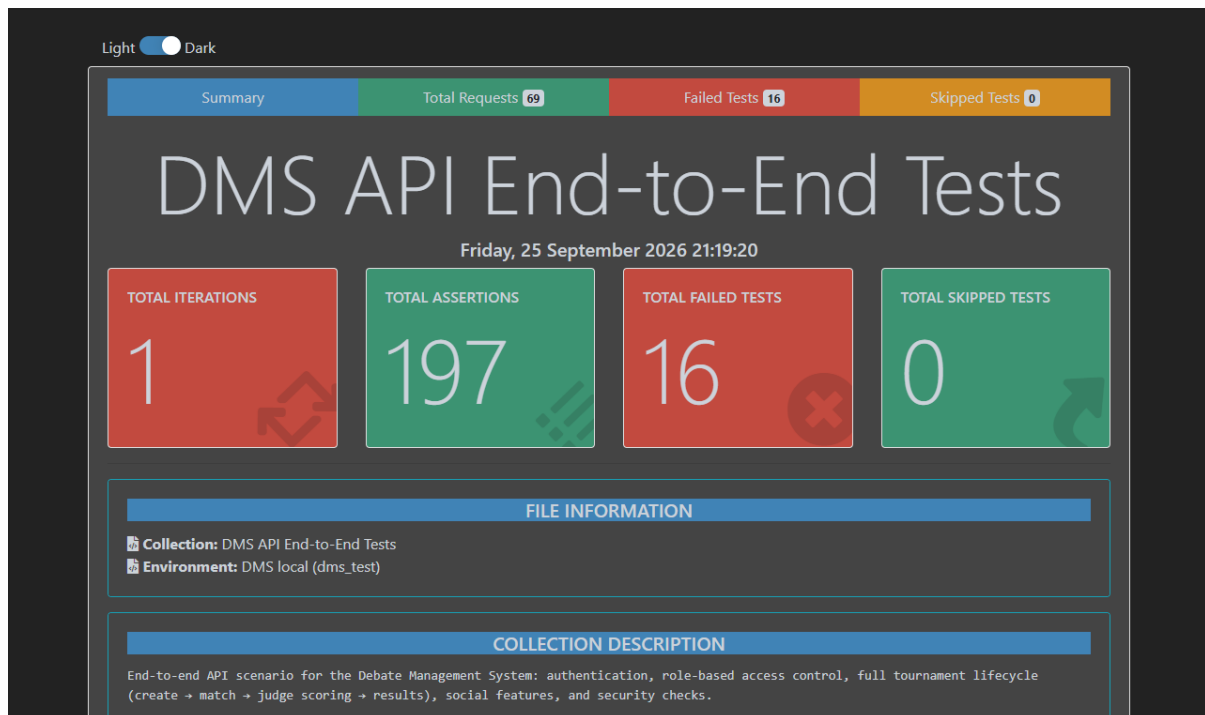


Figure 3. Newman HTML report for the Postman API run

Q Search tests	All 25	✓ Passed 21	✗ Failed 4	Flaky 0	Skipped 0	🔄	⚙️
> Metadata		9/25/2026, 9:29:02 PM Total time: 1.2m					
05-score-sheet.spec.ts							
✗ Judge score sheet (browser) > score sheet shows the organizer's custom criteria							11.4s
05-score-sheet.spec.ts:54							View Trace
✗ Judge score sheet (browser) > after submitting, reopening the sheet shows "already submitted"							11.4s
05-score-sheet.spec.ts:63							View Trace
02-session.spec.ts							
✗ Multi-tab session handling (browser) > an expired token sends the user back to login							15.9s
02-session.spec.ts:38							View Trace
✓ Multi-tab session handling (browser) > logging in as another user in a second tab warns the first tab							1.3s
02-session.spec.ts:11							
✓ Multi-tab session handling (browser) > logging out in a second tab tells the first tab the session ended							1.2s
02-session.spec.ts:23							
04-public.spec.ts							
✗ Public pages (browser) > home page loads live matches for visitors without API errors							1.7s
04-public.spec.ts:24							View Trace
✓ Public pages (browser) > / loads without errors							1.1s
04-public.spec.ts:10							
✓ Public pages (browser) > /news loads without errors							1.1s
04-public.spec.ts:10							
✓ Public pages (browser) > /about loads without errors							1.2s
04-public.spec.ts:10							

Figure 4. Playwright HTML report (failures listed first)

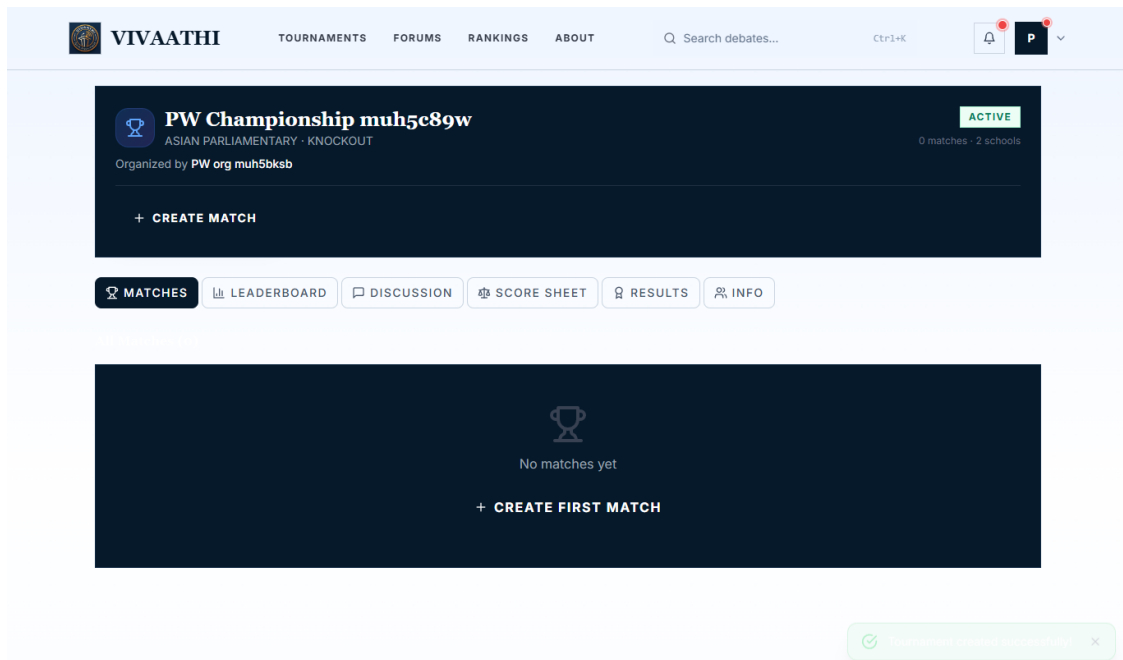


Figure 5. Playwright: tournament created through the 6-step wizard (passing test)

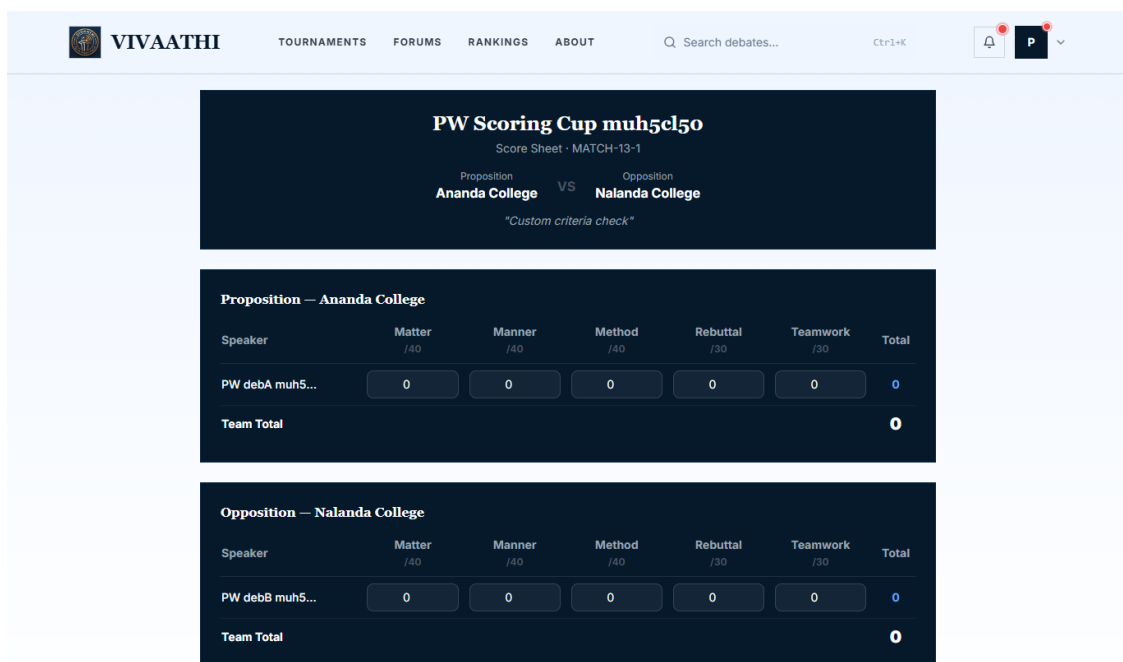


Figure 6. Playwright evidence for DEF-03: the organizer set "Persuasion" and "Evidence", but the judge sees the built-in criteria

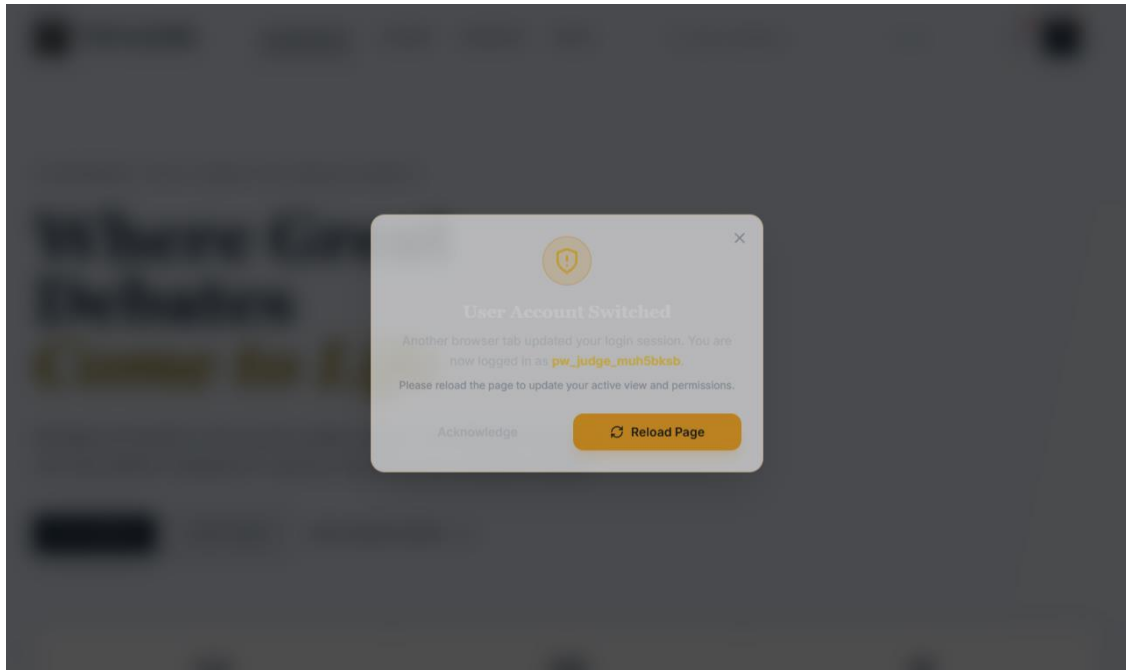


Figure 7. Multi-tab conflict warning works (passing test). Its low-contrast styling is DEF-14

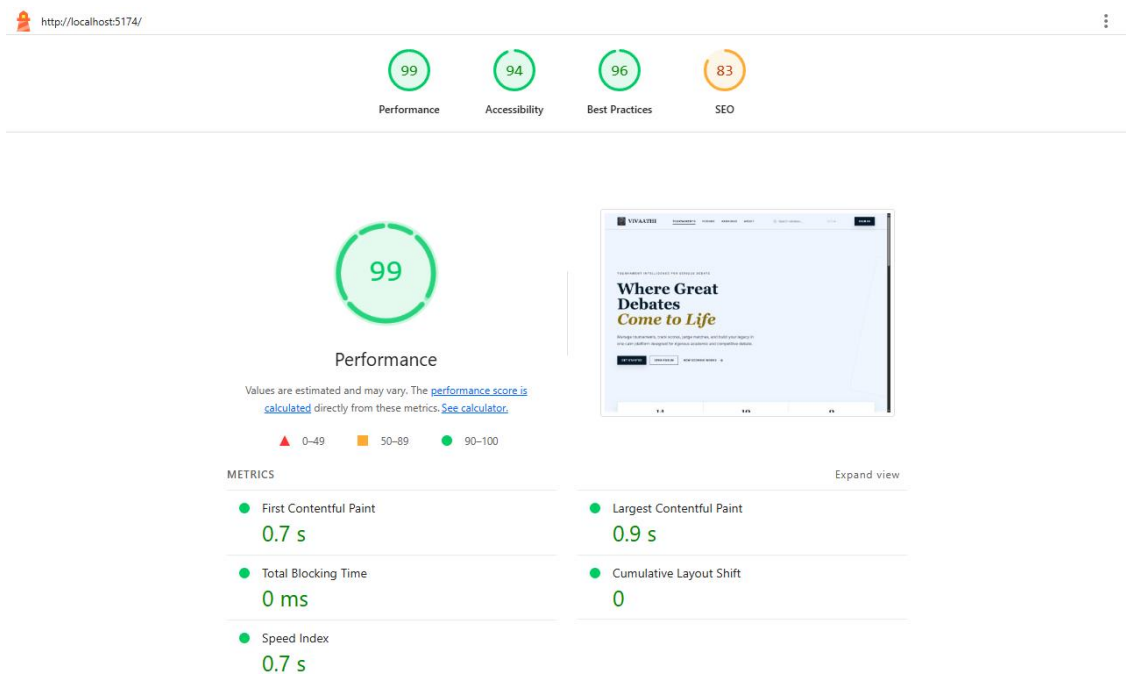


Figure 8. Lighthouse report for the home page

7. Defect log

Testing found 15 defects. None has been fixed yet, because this work only adds tests and does not change application code. Each entry says which test found it and suggests a fix. The same list is in the Excel workbook (sheet "Defect Log") with Status and Assigned-to columns for tracking.

ID	Sev.	Defect	Found by
DEF-01	High	Any logged-in user can edit another user's profile	JUnit OwnershipSecurity · Postman 06
DEF-02	High	Discussion comments can be posted under someone else's name	JUnit OwnershipSecurity · Postman 06
DEF-03	High	Judge's score sheet ignores the organizer's custom criteria	JUnit Workflow · Postman 04 · Playwright 05
DEF-04	Medium	"Already submitted" check on the score sheet never works	JUnit Workflow · Postman 04 · Playwright 05
DEF-05	Medium	Expired or invalid login returns 403 instead of 401, so users are never logged out	JUnit RBAC · Postman 03 · Playwright 02
DEF-06	Medium	An organizer can delete another organizer's tournament	JUnit OwnershipSecurity
DEF-07	Medium	Any logged-in user can delete anyone's discussion comment	JUnit OwnershipSecurity · Postman 06
DEF-08	Medium	Public user list exposes every e-mail address	JUnit OwnershipSecurity · Postman 06
DEF-09	Low	Users can mark other users' notifications as read	JUnit OwnershipSecurity · Postman 06
DEF-10	Low	Role-denied requests return 400 instead of 403	JUnit RBAC · Postman 03/04
DEF-11	Low	Home page always shows 0 live matches to visitors	JUnit RBAC · Postman 01 · Playwright 04 · Lighthouse
DEF-12	Low	/api/auth/me without a token returns 400 (server error) instead of 401	JUnit RBAC · Postman 02
DEF-13	Low	Sign-up accepts invalid e-mail addresses	JUnit Auth · Postman 02
DEF-14	Low	Session warning modal text is hard to read	Playwright screenshot review
DEF-15	Low	Login page accessibility issues	Lighthouse

7.1 Details and suggested fixes

DEF-01 (High, Security): Any logged-in user can edit another user's profile

PUT /api/users/{id} updates whichever id is in the URL; there is no check that it belongs to the caller. Debater A changed Debater B's bio (the profile-picture upload endpoint follows the same pattern).

Suggested fix: Compare the id with the authenticated user (or allow organizers only) before updating.

DEF-02 (High, Security): Discussion comments can be posted under someone else's name

POST /api/discussion takes the author from the "userId" field in the request body instead of the login token.

Suggested fix: Ignore userId in the body; use the authenticated user.

DEF-03 (High, Functional): Judge's score sheet ignores the organizer's custom criteria

GET /api/score-templates/{id} fails with "Type definition error ... ByteBuddyInterceptor" (a raw JPA entity with a lazy proxy is returned). The page swallows the error and silently falls back to built-in criteria.

Suggested fix: Return a DTO (id, name, criteriaJson) instead of the entity.

DEF-04 (Medium, Functional): "Already submitted" check on the score sheet never works

GET /api/score-sheets/{match}/{judge} fails with the same serialization error, so a judge who already submitted sees an empty form again; re-submitting is then rejected by the server.

Suggested fix: Return a DTO for ScoreSheetSubmission.

DEF-05 (Medium, Security / UX): Expired or invalid login returns 403 instead of 401, so users are never logged out

Spring Security has no authentication entry point configured, so missing/expired tokens get 403. The frontend only logs out on 401, so an expired session leaves the user on a page where every request fails.

Suggested fix: Add `.exceptionHandling(e -> e.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))`.

DEF-06 (Medium, Security): An organizer can delete another organizer's tournament

DELETE /api/tournaments/{id} checks the ORGANIZER role but not ownership.

Suggested fix: Allow delete only when tournament.organizer is the caller.

DEF-07 (Medium, Security): Any logged-in user can delete anyone's discussion comment

DELETE /api/discussion/{id} has no ownership check.

Suggested fix: Allow only the author (or the tournament organizer).

DEF-08 (Medium, Privacy): Public user list exposes every e-mail address

GET /api/users is public and returns each user's e-mail to anonymous visitors.

Suggested fix: Use a public profile DTO without e-mail, or require login.

DEF-09 (Low, Security): Users can mark other users' notifications as read

PUT /api/notifications/{id}/read receives the username but never checks it.

Suggested fix: Check `notification.user == caller`.

DEF-10 (Low, API): Role-denied requests return 400 instead of 403

GlobalExceptionHandler catches RuntimeException, which includes Spring's AccessDeniedException, and turns "Access Denied" into 400. Access is still correctly blocked.

Suggested fix: Add an `@ExceptionHandler(AccessDeniedException.class)` returning 403.

DEF-11 (Low, Functional): Home page always shows 0 live matches to visitors

The public home page calls GET /api/matches/live, which is not in the public list in SecurityConfig, so visitors get 403 (also logged as a console error by Lighthouse).

Suggested fix: Permit GET /api/matches/** for everyone.

DEF-12 (Low, API): /api/auth/me without a token returns 400 (server error) instead of 401

/api/auth/** is fully public, so the controller runs with no Authentication and throws a NullPointerException.

Suggested fix: Permit only /api/auth/login and /signup; require auth for /me.

DEF-13 (Low, Validation): Sign-up accepts invalid e-mail addresses

SignupRequest has no validation annotations, so "not-an-email" is stored.

Suggested fix: Add @Valid with @Email/@NotBlank/@Size on SignupRequest.

DEF-14 (Low, UI): Session warning modal text is hard to read

SessionChangedModal still uses the old dark-theme classes (glass-dark, text-white), so after the UI refresh the title and message are white on a light panel.

Suggested fix: Restyle the modal with the new theme colors.

DEF-15 (Low, Accessibility): Login page accessibility issues

The show/hide password button has no accessible name; some text fails color-contrast; pages have no <main> landmark or meta description.

Suggested fix: Add aria-label to icon buttons, a <main> element and a meta description.

7.2 Other observations

- The default JWT secret and a database password are committed in application.properties. Anyone who reads the repository could forge login tokens if production ever runs with the default secret. Set JWT_SECRET and DB_PASSWORD only as environment variables.
- The home page lists the first 6 tournaments (oldest first), so new tournaments disappear from it as the list grows.
- Generating the next knockout round silently drops a winner when there is an odd number of winners.
- The one existing frontend test (LoginPage.test.tsx) had been broken since the UI redesign changed the password placeholder. It was rewritten as part of this work.

7.3 Bug resolution history

Bugs fixed during development, taken from the project's git history. The last column shows which new test now protects each fix against regression.

Commit	Date	Author	Problem fixed	Now covered by
c045431	2026-09-21	Pathi-02	Multiple log-ins in one browser conflicted	Regression-tested: AuthContext.test.tsx and Playwright 02 (passing)
c0a3d93	2026-09-21	Pathi-02	No warning when session ended in another tab	Regression-tested: AuthContext.test.tsx and Playwright 02 (passing)
eebd145	2026-09-22	Pathi-02	Notification bugs	Notification flow covered by JUnit Workflow and Postman 04/05
dec7edb	2026-07-10	Jeyanth3	Tournament creation issue	Wizard covered by Playwright 03 (passing)
b1a77ed	2026-07-09	Jeyanth3	Submit button issue	Score submission covered by Playwright 03 (passing)
9d3ff79	2026-07-09	Jeyanth3	Profile issue	Profile update covered by JUnit OwnershipSecurity
e7c7f7e	2026-05-02	Jeyanth3	Organizer and judge dashboard issue	Dashboard access covered by App.test.tsx and Playwright 01
980dcf4	2026-05-02	Jeyanth3	Viewer-side dashboard and notification access	Covered by App.test.tsx route rules
87645b6	2026-05-02	Jeyanth3	Debater search menu UI bug	Debater search used in Playwright 03 wizard (passing)

Commit	Date	Author	Problem fixed	Now covered by
c655b0d	2026-05-01	Jeyanth3	Login issue	Covered by LoginPage.test.tsx and Playwright 01 (passing)
e18de01	2026-06-13	Jeyanth3	TypeScript build errors for Vercel	Build check: vite build succeeds

8. Manual testing

The workbook DMS_Test_Workbook.xlsx contains 40 manual test cases covering every module. Each has preconditions, steps, the expected result, a Status dropdown (Pass / Fail / Blocked / Not run), tester and date columns, and a note on which automated test covers the same behavior. Team members should run these in the browser before the presentation and fill in the results; any failure goes in the Defect Log sheet.

9. How to run the tests

Suite	Command (from repository root)	Report
Backend unit + integration	cd backend && mvn test	backend/target/site/jacoco/index.html
Frontend	cd frontend && npx vitest run --coverage	frontend/test-results/coverage/index.html
API end-to-end	cd testing && npm run api (backend on :8081 with DB_NAME=dms_test)	testing/reports/newman/api-report.html
Browser end-to-end	cd testing && npx playwright test	testing/reports/playwright/html/index.html
Regenerate this report	cd testing && node report/collect.js && node report/build-docx.js	testing/reports/DMS_Testing_Report.docx

A GitHub Actions workflow (.github/workflows/tests.yml) runs the backend, frontend and Postman suites on every push. It will show red until the defects above are fixed, which is intended: the pipeline turns green as fixes land.

10. Limitations and next steps

- Fix the three high-severity defects first (DEF-01, DEF-02, DEF-03). Then re-run all suites; the matching tests will turn green.
- Coverage is lower in controllers (65%) because several read-only endpoints are exercised only by the Postman run, which JaCoCo does not measure.
- Load and stress testing (for example with k6 or JMeter) and testing against the deployed Azure/Vercel environment were not done.
- Real user-acceptance testing with debaters, judges and organizers is still recommended.